1. Amazon ElastiCache — Console Dashboard Wise Important Interview Questions & Answers

```
ElastiCache
│
├── Dashboard
│
├── Serverless Caches
│
├── Caches
│   └── Individual Cache
│       ├── Connectivity
│       ├── Security
│       ├── Monitoring
│       ├── Logs
│       ├── Configuration
│       └── Maintenance
│
├── Subnet Groups
│
├── Parameter Groups
│
├── User Groups
│
├── Users
│
├── Reserved Nodes
│
└── Events
```

Dashboard

1. What is Amazon ElastiCache?

Answer:

Amazon ElastiCache is a managed in-memory caching service provided by AWS.
It supports Redis/Valkey and Memcached engines for storing frequently accessed data in memory.
It helps applications reduce database load and improve response time.

For example:

Suppose our application frequently reads product information from RDS.
We can store frequently used product data in ElastiCache so the application can retrieve it faster.
This reduces repeated requests to the database.

Example:

Application
↓
ElastiCache
↓
Cached Data
↓
RDS

Simple ga gurthupettuko:

ElastiCache = Fast in-memory cache

Serverless Caches

2. What is ElastiCache Serverless?

Answer:

ElastiCache Serverless provides a managed cache without requiring us to manage cache nodes manually.
AWS automatically handles capacity based on the application's workload.
It is useful when traffic changes frequently and we want simpler cache management.

For example:

Suppose our application has normal traffic during the day but suddenly receives heavy traffic during a sale.
Serverless ElastiCache can automatically adjust capacity based on demand.
We don't need to manually add or remove cache nodes.

Example:

Application
↓
ElastiCache Serverless
↓
Automatic Capacity
↓
Cached Data

Simple ga gurthupettuko:

Serverless Cache = AWS manages cache capacity

Caches

3. What is an ElastiCache Cache?

Answer:

A cache is the actual ElastiCache deployment where our application stores frequently accessed data in memory.
Depending on the configuration, it can use Redis/Valkey or Memcached.
Applications connect to the cache using its endpoint.

For example:

Suppose an application frequently requests user session information.
We can store those sessions in an ElastiCache cache instead of querying the database every time.
The application gets the information much faster.

Example:

Application
↓
Cache Endpoint
↓
ElastiCache
↓
User Session

Simple ga gurthupettuko:

Cache = In-memory data store used by application

Individual Cache → Connectivity

4. What is the ElastiCache endpoint?

Answer:

The endpoint is the network address that an application uses to connect to an ElastiCache deployment.
The application uses the appropriate endpoint to send cache commands and retrieve data.
The endpoint depends on the cache configuration and engine.

For example:

Suppose our application needs to connect to a Redis cache.
We configure the application's cache connection using the ElastiCache endpoint and required port.
The application can then read and write cached data.

Example:

Application
↓
ElastiCache Endpoint
↓
Redis/Valkey Cache

Simple ga gurthupettuko:

Endpoint = Address used to connect to cache

---

5. How does ElastiCache work inside a VPC?

Answer:

ElastiCache resources are normally deployed inside a VPC so applications can communicate with them privately.
We configure networking using subnets, subnet groups, and security groups.
This keeps cache traffic away from direct public internet access.

For example:

Suppose EC2 and ElastiCache are running inside the same VPC.
The EC2 application connects to the ElastiCache private endpoint through the configured security rules.
The cache does not need to be publicly accessible.

Example:

VPC
│
├── EC2
│ ↓
│ Security Group
│ ↓
└── ElastiCache

Simple ga gurthupettuko:

ElastiCache = Private cache inside VPC

Individual Cache → Security

6. How is ElastiCache secured?

Answer:

ElastiCache security can be controlled using VPC networking, security groups, authentication, and encryption.
Security groups control which clients can connect to the cache.
For supported configurations, encryption protects data in transit and at rest.

For example:

Suppose only our application servers should access the cache.
We can allow the application security group to connect to the ElastiCache security group on the required cache port.
Other unwanted network connections can be blocked.

Example:

EC2 Security Group
↓
Allowed Connection
↓
ElastiCache Security Group
↓
Cache

Simple ga gurthupettuko:
Security = VPC + Security Group + Authentication + Encryption

---

7. What is authentication in ElastiCache?

Answer:
Authentication controls which clients are allowed to connect to supported ElastiCache engines.
For Redis/Valkey, AWS provides authentication mechanisms such as AUTH and RBAC through users and user groups.
This adds an additional security layer beyond network access.

For example:
Suppose an application reaches the Redis cache through the correct security group.
The application may still need valid authentication credentials before it can access the cache.
This prevents unauthorized clients from using the cache.

Example:
Application
↓
Network Access
↓
Authentication
↓
ElastiCache

Simple ga gurthupettuko:
Authentication = Verify who can access cache

Individual Cache → Monitoring

8. How do you monitor ElastiCache?

Answer:
ElastiCache provides monitoring metrics through Amazon CloudWatch.
We can monitor metrics such as CPU usage, memory usage, connections, cache hits, and cache misses.
These metrics help us identify performance and capacity problems.

For example:
Suppose cache memory usage becomes very high.
We can check CloudWatch metrics and determine whether the cache needs resizing or configuration changes.
We can also create alarms for important metrics.

Example:
ElastiCache
↓
CloudWatch Metrics
↓
CPU / Memory / Connections
↓
Alarm

Simple ga gurthupettuko:
Monitoring = ElastiCache metrics through CloudWatch

---

9. What are Cache Hits and Cache Misses?

Answer:
A cache hit happens when the requested data is already available in the cache.
A cache miss happens when the requested data is not available, so the application usually retrieves it from the database or another source.
A higher cache-hit rate generally means the cache is serving more requests successfully.

For example:
Suppose the application requests product ID 101 and that data already exists in ElastiCache.
That request is a cache hit and the application gets the data quickly.
If the data is not present, it becomes a cache miss and the application may query RDS.

Example:
Request
↓
ElastiCache
├— Hit → Return Data
└— Miss → RDS → Return Data

Simple ga gurthupettuko:
Hit = Data found in cache
Miss = Data not found in cache

Subnet Groups

10. What is an ElastiCache Subnet Group?

Answer:

An ElastiCache subnet group is a collection of subnets that can be used for an ElastiCache deployment.
The subnets are normally selected from multiple Availability Zones within a VPC.
It tells ElastiCache where cache resources can be placed in the network.

For example:

Suppose our VPC has private subnets in two Availability Zones.
We create an ElastiCache subnet group containing those private subnets.
We then use that subnet group when creating the cache.

Example:

VPC
↓
ElastiCache Subnet Group
├— Private Subnet-A → AZ-1
└— Private Subnet-B → AZ-2

Simple ga gurthupettuko:

Subnet Group = Where ElastiCache can run in VPC

Parameter Groups

11. What is an ElastiCache Parameter Group?

Answer:

A parameter group contains configuration settings used by an ElastiCache engine.
It allows us to customize supported engine parameters without changing the application code.
Different cache deployments can use different parameter groups based on their requirements.

For example:

Suppose we need a specific Redis/Valkey configuration for our application.
We can create or modify an appropriate parameter group and associate it with the cache.
The cache then uses those configuration parameters.

Example:

Parameter Group
↓
Engine Parameters
↓
ElastiCache Cache

Simple ga gurthupettuko:

Parameter Group = Cache configuration settings

Users

12. What are ElastiCache Users?

Answer:

ElastiCache users define identities that can authenticate to supported Redis/Valkey caches.
A user can have authentication credentials and permissions associated with it.
Users help control which operations different clients are allowed to perform.

For example:

Suppose one application needs read-only access while another application needs read and write access.
We can create users with appropriate permissions for those applications.
This gives us more controlled access to the cache.

Example:

Application
↓
ElastiCache User
↓
Permissions
↓
Redis/Valkey

Simple ga gurthupettuko:

User = Identity used to access cache

User Groups

13. What is an ElastiCache User Group?

Answer:

A user group is a collection of ElastiCache users that can be associated with a Redis/Valkey cache.
It allows us to manage access for multiple users together.
Each user can have permissions based on the configured access rules.

For example:

Suppose we have multiple application users that need access to the same Redis cache.
Instead of managing every user separately at the cache level, we can organize them into a user group.
This makes access management easier.

Example:

User Group

├— User-A
├— User-B
└— User-C
↓
Redis/Valkey Cache
Simple ga gurthupettuko:
User Group = Group of cache users

Reserved Nodes
14. What are ElastiCache Reserved Nodes?
Answer:
Reserved Nodes provide a pricing option where we commit to using a specified ElastiCache node configuration for a term.
In return, we can receive discounted pricing compared with equivalent On-Demand usage, depending on the reservation.
They are useful when we have predictable long-term cache usage.
For example:
Suppose our production application continuously runs the same ElastiCache node configuration throughout the year.
Instead of paying entirely at On-Demand rates, we can consider Reserved Nodes for predictable workloads.
This can help reduce long-term cache costs.
Example:
Predictable Workload
↓
Reserved Nodes
↓
Long-Term Commitment
↓
Potential Cost Savings
Simple ga gurthupettuko:
Reserved Nodes = Long-term predictable cache usage + pricing benefit

---

Events
15. What are ElastiCache Events?
Answer:
ElastiCache events provide information about important changes or activities affecting cache resources.
They can help us understand events such as maintenance, configuration changes, or other cache-related activities.
They are useful for troubleshooting and operational monitoring.
For example:
Suppose an ElastiCache deployment undergoes maintenance.
We can check the Events section to understand what happened and when it happened.
This helps us investigate unexpected changes or availability issues.
Example:
ElastiCache
↓
Events
↓
Maintenance / Changes / Status
↓
Troubleshooting
Simple ga gurthupettuko:
Events = Important activities happening to ElastiCache

Most Important Interview Differences
16. What is the difference between Redis/Valkey and Memcached?
Answer:
Redis/Valkey provides richer features such as persistence, replication, and more advanced data structures.
Memcached is a simpler in-memory caching system mainly designed for straightforward key-value caching.
The choice depends on application requirements and the features we need.
For example:
If our application needs advanced caching capabilities and features such as replication, Redis/Valkey may be suitable.
If we only need a simple distributed cache, Memcached can be considered.
We choose the engine based on the workload.
Example:
Application
↓
Cache Requirement
├— Advanced Features → Redis/Valkey
└— Simple Cache → Memcached
Simple ga gurthupettuko:
Redis/Valkey = Advanced cache
Memcached = Simple cache

---

17. What is the difference between ElastiCache and RDS?
Answer:
ElastiCache is an in-memory cache used mainly to improve application performance.
RDS is a managed relational database used for persistent structured data.
ElastiCache normally works alongside a database rather than replacing it.
For example:
Suppose our application stores customer information permanently in RDS.
Frequently accessed customer data can also be stored temporarily in ElastiCache for faster reads.
If the cache misses, the application can retrieve the data from RDS.
Example:
Application
↓
ElastiCache → Fast Cached Data
↓
RDS → Persistent Data
Simple ga gurthupettuko:
RDS = Persistent Database
ElastiCache = Fast Temporary Cache